

CRM Application

Full-Stack Internship Assessment — Final Project Report

Author: Abdullah Hafi | Date: May 8, 2026 | GitHub: github.com/AbdullahHafi/crm_system

1. Introduction

This report documents the development of a Customer Relationship Management (CRM) web application built as part of a full-stack internship assessment. The application enables users to manage sales leads through a complete CRUD interface, track deal values, add notes, and view pipeline analytics via a dashboard.

2. Technology Stack

Layer	Technology	Purpose
Frontend	React 18 (Vite)	UI library & build tool
Routing	React Router v6	Client-side navigation
Styling	Tailwind CSS 3	Utility-first CSS framework
Icons	Lucide React	Modern icon library
HTTP Client	Axios	API requests
Backend	Node.js + Express	REST API server
Database	Supabase (PostgreSQL)	Cloud-hosted relational DB
Authentication	JWT + bcryptjs	Token-based auth with hashed passwords

3. Features Implemented

3.1 Authentication

- JWT-based login with email & password
- Passwords hashed using bcryptjs

- Protected routes via auth middleware (frontend & backend)
- Persistent sessions using localStorage tokens

3.2 Dashboard

- Real-time statistics: Total Leads, New, Qualified, Won, Lost
- Total Pipeline Value and Total Won Value cards
- Currency-formatted numbers (USD)

3.3 Lead Management (CRUD)

- **Create** — Add new leads with name, company, email, phone, source, salesperson, status, and deal value
- **Read** — View all leads in a table; view individual lead details
- **Update** — Edit any lead field via a pre-populated form
- **Delete** — Remove leads with confirmation dialog

3.4 Lead Notes

- Add timestamped notes to any lead
- Notes display author email and creation time
- Cascading delete when parent lead is removed

3.5 Search & Filtering (Bonus)

- Search by lead name, company name, or email
- Filter by status (New, Contacted, Qualified, Proposal Sent, Won, Lost)
- Filter by lead source and assigned salesperson

4. Project Architecture

4.1 Folder Structure

```
crm_system/
├── backend/
│   ├── controllers/
│   │   ├── authController.js
│   │   ├── leadController.js
│   │   └── noteController.js
│   ├── middleware/
│   │   └── authMiddleware.js
│   ├── routes/
│   │   ├── authRoutes.js
│   │   ├── leadRoutes.js
│   │   └── noteRoutes.js
│   ├── db.js
│   ├── server.js
│   ├── setup.sql
│   └── package.json
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── Layout.jsx
│   │   │   └── Navbar.jsx
│   │   ├── context/
│   │   │   └── AuthContext.jsx
│   │   ├── pages/
│   │   │   ├── Login.jsx
│   │   │   ├── Dashboard.jsx
│   │   │   ├── Leads.jsx
│   │   │   ├── LeadDetails.jsx
│   │   │   └── CreateEditLead.jsx
│   │   ├── api.js
│   │   ├── App.jsx
│   │   └── main.jsx
│   └── package.json
├── .gitignore
└── README.md
```

4.2 Database Schema

Table	Key Columns	Description
-------	-------------	-------------

users	id (UUID PK), email, password_hash, created_at	Stores registered users
leads	id (UUID PK), lead_name, company_name, email, phone_number, lead_source, assigned_salesperson, status, estimated_deal_value, created_at, updated_at	Stores all sales leads
notes	id (UUID PK), lead_id (FK → leads), note_content, created_by, created_at	Timestamped notes per lead

4.3 API Endpoints

Method	Endpoint	Description	Auth
POST	/api/auth/login	User login, returns JWT	No
GET	/api/auth/me	Get current user info	Yes
GET	/api/leads/dashboard	Get dashboard statistics	Yes
GET	/api/leads	List all leads (with search/filter)	Yes
GET	/api/leads/:id	Get single lead details	Yes
POST	/api/leads	Create a new lead	Yes
PUT	/api/leads/:id	Update existing lead	Yes
DELETE	/api/leads/:id	Delete a lead	Yes
GET	/api/notes/:leadId	Get notes for a lead	Yes
POST	/api/notes	Add a note to a lead	Yes

5. Setup & Running Instructions

- Prerequisites:** Node.js v18+, a Supabase account
1. Run `backend/setup.sql` in the Supabase SQL Editor to create tables
 2. Configure `backend/.env` with `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`, `JWT_SECRET`
 3. Run `cd backend && npm install && npm run dev`
 4. Run `cd frontend && npm install && npm run dev`
 5. Open `http://localhost:5173` — Login with **admin@example.com / password123**

6. Application Pages

Page	Route	Description
Login	/login	Glassmorphism-styled login form with gradient background
Dashboard	/	Overview cards showing lead stats and pipeline values
Leads List	/leads	Searchable & filterable table of all leads
Lead Details	/leads/:id	Full lead info, notes section, edit/delete actions
Create Lead	/leads/new	Form to add a new lead
Edit Lead	/leads/:id/edit	Pre-populated form to update a lead

7. Lead Status Lifecycle

New → Contacted → Qualified → Proposal Sent → Won / Lost

8. Security Measures

- Passwords hashed with bcryptjs (10 salt rounds) — never stored in plaintext
- JWT tokens with 1-day expiry for session management

- Backend auth middleware protects all lead and note endpoints
- Frontend PrivateRoute component prevents unauthorized page access
- .env files excluded from version control via .gitignore
- CORS enabled on the backend for cross-origin requests

9. Reflection & Challenges

This CRM application implements all core requirements (authentication, CRUD for leads, dashboard analytics) along with bonus features (search, multi-filter, lead notes). Key challenges included:

- Configuring Supabase service role keys for secure server-side database access
- Implementing bcrypt password verification between the seeded hash and user input
- Designing a clean, modern UI with responsive layouts using Tailwind CSS
- Managing authentication state across the React app using Context API

10. Conclusion

The CRM application is a fully functional full-stack web application that demonstrates proficiency in React, Node.js/Express, and Supabase (PostgreSQL). It provides a modern, responsive interface for managing sales leads with real-time filtering, note-taking capabilities, and a clean dashboard for pipeline analytics.